Lecture 1 · Foundations / SDLC / Git

SDLC: พัฒนาระบบ ไม่ใช่แค่เขียนโค้ด

Software engineering: พัฒนาระบบให้แก้ปัญหาผู้ใช้ ทดสอบได้ ปลอดภัยและดูแลต่อได้; โค้ดรันได้ ≠ ผู้ใช้ยอมรับ Planning → Requirements → Specification → Design → Implementation → Testing → Deployment → Maintenance **Planning**: เหตุผล ขอบเขตและความเสี่ยง; **Req**: ต้องการอะไร; **Spec**: ตกลงพฤติกรรม; **Design**: ทำอย่างไร; **Test**: พิสูจน์ผล; **Deploy**: ตั้งค่า นำขึ้นใช้ เฝ้าดู/ย้อนรุ่น; **Maintain**: แก้และปรับระบบ **คำอธิบายเสริม:** Feasibility: เทคนิคพร้อม คุ้มทุน ใช้ได้จริง ทันเวลา/ทรัพยากรพอไหม

Process models / Agile values

Model	จุดเด่น / ข้อควรระวัง
Waterfall	แผนและ sign-off ชัด; เหมาะกับ req คงที่ แต่ feedback ช้า เปลี่ยนแพง
Incremental	ส่ง vertical slice ที่ใช้ได้ทีละส่วน; คุม architecture และ integration
Iterative	ปรับ solution เดิมซ้ำตาม feedback
Scrum	sprint goal, timebox, increment; ตรวจสอบและปรับตัว
Kanban	งานไหลต่อเนื่อง; จำกัด WIP วัด cycle time/ throughput

Agile ให้ความสำคัญมากกว่า: คนและปฏิสัมพันธ์ > กระบวนการ/เครื่องมือ; ซอฟต์แวร์ที่ใช้ได้ > เอกสารครบถ้วน; ร่วมมือกับลูกค้า > ต่อรองสัญญา; ตอบรับการเปลี่ยนแปลง > ยึดแผน ด้านขวายังมีคุณค่า **Vertical slice**: UI + API + DB + test ใช้ได้ครบเส้นทาง; ไม่ใช่ทำ DB อย่างเดียวแล้วค่อยต่อ UI **Definition of Done (DoD)**: เกณฑ์แสร้งว่าม เช่น ผ่าน AC/tests/review แก้ข้อบกพร่องสำคัญ เอกสารตรง และรวมงานตามกติกา

AI engineering / SDD + TDD

Agent: observe → plan → tool/edit/test → inspect → adjust; ต่างจาก chatbot ที่เน้นตอบข้อความ **Prompt ที่ชัด**: ให้บริบท ข้อกำหนด/contract ข้อจำกัด ขอบเขตไฟล์ วิธีทดสอบและหลักฐาน; แบ่งงานก่อนทำ แล้วตรวจ diff กับผลจริง คนรับผิดชอบเป้าหมาย การยอมรับและความเสี่ยง; คำว่า “ผ่าน” จาก agent ไม่ใช่หลักฐาน ระวัง hallucination, regression, secrets, dependencies และ docs ที่ล้าสมัย

Architecture / Git พื้นฐาน

React UI → HTTP API/Express → service/กฎธุรกิจ → Prisma ORM → PostgreSQL; UI ไม่ต่อฐานข้อมูลโดยตรง **Working tree** แก้ไฟล์ → `add` เลือก snapshot เข้า **index** → `commit` บันทึกใน local → `push` ส่งไป remote Issue → branch → commit → PR → review → staging → main; feature แยกงาน; staging รวม/ทดสอบ; main คืองานที่ยอมรับ; README บอก setup/run/test/env/กติกา

```
git status; git diff; git diff --cached
git log --oneline --graph --decorate --all
```

ไม่เก็บ .env, credentials, node_modules หรือ temp; ใช้ .env.example และระบุชื่อไฟล์ตอน `git add`

Lecture 2 · Requirement Engineering

ระดับข้อกำหนด: Business = เป้าหมาย; User/URS = ผู้ใช้ต้องการอะไร/ทำไม; System/SysRS = รวมฮาร์ดแวร์ ส่วนเชื่อมต่อ/กฎระเบียบ; Software/SRS = พฤติกรรมและข้อจำกัดซอฟต์แวร์

ชนิด	คำถาม / ตัวอย่าง
FR	Functional requirement: ระบบทำอะไร
NFR	Non-functional requirement: คุณภาพระดับใด ภายใต้เงื่อนไขใด
BR/ domain	Business/domain rule: กฎธุรกิจ เช่น ไม่เกิน 3 ครั้ง/ภาค
Constraint	ข้อจำกัดทางเลือก เช่น เทคโนโลยีหรือที่ตั้ง server

FR/NFR/BR เป็นชนิด ไม่ใช่ระดับ user/system; กฎหนึ่งผูกกับหลาย FR ได้ ไม่ต้องแยก BR ให้ทุกข้อ **NFR**: performance, security, usability, reliability, maintainability; ระบุตัวชี้วัดเงื่อนไขและเกณฑ์ผ่าน

RE process / Quality checklist

Elicit/analyse → specify → validate → manage changes; หาผู้เกี่ยวข้อง/เป้าหมาย → สอบถาม สังเกต อ่านเอกสาร → บันทึก → ยืนยัน → จัดลำดับ **Fact**: ระบุแล้ว; **Assumption**: ยังไม่ยืนยัน; **Question**: ต้องถาม; แยกสิ่งที่ทำ/ไม่ทำ/ไว้ภายหลัง **Quality**: ชัดเจน ไม่กำกวม มีเรื่องเดียว จำเป็น ครบ สอดคล้อง ทำได้จริง ทดสอบและย้อนรอยได้; ระบุ actor/action/object/condition **Validation**: ทบทวน สร้างต้นแบบหรือออกแบบ tests เพื่อตรวจความเข้าใจ; **Change control**: บันทึกเหตุผล เจ้าของ ผลกระทบ ข้อสรุปและรุ่น

Proposal 8 ช่อง / Evidence chain

Problem · Users · Solution · Expected value · Scope · Risk · Effort · Evidence of completion แยกสิ่งที่มีจากพร้อมหลักฐาน; ข้อเสนอเพิ่มต้องติดป้าย ไม่เขียนเป็นข้อเท็จจริง Req ID/ source → feature/design → Issue/code → AC/test → PR/ release → demo; ต้องชี้ไฟล์และผลจริง **Acceptance criteria (AC)**: เกณฑ์ยอมรับเฉพาะงาน; **DoD**: เกณฑ์แสร้งว่าม; **Definition of Ready (DoR)**: พร้อมเริ่ม ≠ เสร็จ; MoSCoW: Must / Should / Could / Won't (รอบนี้)

วิเคราะห์ข้อกำหนดพร้อมเหตุผล

ปัญหา	ต้องซื้อระไร / แก้อย่างไร
กำกวม	“quick/easy” ไม่มีเกณฑ์ → ตกลง metric/load/ threshold
ไม่ atomic	หลาย actor/หลายเป้าหมาย → แยกข้อ แต่คง trace กลับต้นฉบับ
ไม่ครบ	ขาด precondition/output/error/สิทธิ์ → ถามแต่ละสถานะ
ขัดแย้ง	เช่น limit 3 กับ 4 → ให้ผู้มีอำนาจตัดสิน ไม่เลือกเอง
ตรวจไม่ได้	แปลงคำกว้างเป็นผลสังเกตได้ พร้อมวิธีวัด
ทำไม่ได้	ตรวจเทคนิค/เวลา/งบ; เจรจาขอบเขต ไม่รับปากลอย ๆ
ย้อนกลับไม่ได้	ใส่ ID/source/owner แล้วโยง AC/design/test

Elicitation: ใครทำ? เมื่อไร? ข้อมูลเข้า/ออก? กฎ/สิทธิ์? ข้อยกเว้น? นอกขอบเขต? ใช้ interview/observation/docs/ prototype ตรวจความเข้าใจว่าม **BR** → **FR**: จำกัด 3 รายการ/ภาค → ระบบปฏิเสธรายการที่ 4; NFR ระบุระดับคุณภาพ **คำอธิบายเสริม:** AC: Given มี 3 รายการในภาคนี้; When ขอเพิ่ม; Then ปฏิเสธ/ยังเป็น 3; ถามว่าการยกเลิกนับไหม

Lecture 3 · Specifications / SDD / Tests

SRS → SDS → STS

SRS: ต้องการอะไร; **SDS**: ออกแบบอย่างไร; **STS**: ตรวจอะไร ด้วยข้อมูลใด คาดผลอย่างไร และเก็บหลักฐานใด SRS → feature inventory → req-feature map → system SDS → sprint vertical slice → feature contract → implement → test/ review → integrate → main **System SDS**: โครงสร้าง/stack/ data, auth, มาตรฐาน API/UI/errors และ NFR ร่วม **Feature SDS**: ขอบเขต/req, workflow/state/BR, data/API/UI, สิทธิ์/ validation/errors, AC/tests **Contract 7**: inputs → validations → behavior → persistence → side effects → errors → outputs Invoice: รับ customer/date/lines → ตรวจ customer, qty > 0, price ≥ 0 → คำนวณ → บันทึกหัว/รายการใน transaction เดียว; ผิด → ไม่บันทึก **คำอธิบายเสริม:** receipt เป็น side effect ที่ต้องตกลง

Tests / Red-Green-Refactor

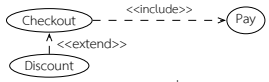
RED: test ล้มเหลวด้วยเหตุผลที่ตั้งใจ → GREEN: เขียนโค้ดเท่าที่จำเป็นให้ผ่าน → REFACTOR: ปรับโครงสร้างโดย tests ยังผ่าน **AAA**: Arrange เตรียมข้อมูล → Act เรียกใช้ → Assert ตรวจผล; ครอบคลุมปกติ/ผิดเงื่อนไข/ขอบเขต/error/สิทธิ์; ต้องจับ defect ได้ ไม่เขียนตาม implementation อย่างเดียว **STS row**: test ID + Req/AC → precondition/data → steps/input → expected UI/API/DB → file + final result **คำอธิบายเสริม:** ค่าขอบเขต (boundary): คะแนน 0-100 → ทดสอบ −1/0/100/101; 0 และ 100 ผ่าน ส่วน −1 และ 101 ถูกปฏิเสธ

ระดับ/ชนิด	ตรวจอะไร
Unit	function/class/component; แยก deps ด้วย mock
Integration	modules/API/DB/interfaces ทำงานร่วมกันจริง
System	ระบบรวมครบ SRS ทั้ง FR/NFR
E2E	เส้นทางผู้ใช้ผ่าน UI → API → DB/external
UAT	ผู้ใช้/ธุรกิจยอมรับตาม need/AC และ sign-off
Regression	ทดสอบของเดิมหลังแก้ไข; ใช้ได้ทุกระดับ
Smoke	เปิดระบบและใช้งานหลักได้หลัง setup/release
Special	load/performance, security, compatibility, recovery

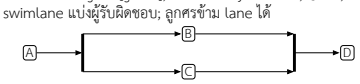
Verification: ทำตรง spec ไหม; **Validation**: ตรงความต้องการผู้ใช้ไหม; coverage สูง ≠ tests จับข้อผิดพลาดได้ดี **Sync**: รอผลตอบ; **Async**: ส่งงานแล้วรับผลภายหลัง; ระบุการลองซ้ำ (retry) และ idempotency: ส่งซ้ำแล้วไม่เกิดผลซ้ำเกินที่ตกลง **POS recovery/outbox**: เก็บ payment attempt; ผลไม่ชัดตรวจ PSP ก่อนเก็บเงินซ้ำ; บันทึก sale + pending inventory แบบ atomic; ส่ง async รอ ack/retry ใช้ transaction ID กัน stock ซ้ำ **BDD**: เริ่มจากพฤติกรรมด้วย Gherkin (outside-in); **TDD**: ให้ tests นำการออกแบบ (inside-out); ใช้ร่วมกันได้

Lecture 4 · UML / Feature Design / UI

Use case: เป้าหมายของ actor ภายใน system boundary; วงรี = goal; actor association เส้นทึบ ไม่มีหัวลูกศร include: base → ขั้นตอนบังคับ (เส้นประ); extend: กรณีเสริม → base (เส้นประ + condition); generalization: child → parent (สามเหลี่ยมโปร่ง)

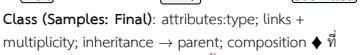


Activity: workflow ครบทางเลือก; ● เริ่ม, action มุมมน, ◇ decision/merge + [guard], bar = fork/join ขนาน, ⊙ จบ; swimlane แบ่งผู้รับผิดชอบ; ลูกศรข้าม lane ได้



fork: ทำ B/C ขนาน; join: รอครบทั้งคู่

Sequence: scenario หนึ่งใน workflow; participant/lifeline อยู่ด้านบน เวลาลงล่าง; call เส้นทึบ/return เส้นประ; activation คือช่วงทำงาน; alt [condition], opt, loop; แสดงระบบภายนอก และผลข้างเคียง **State**: สถานะของ object เดียว; event [guard] / effect; ระบุจุดเริ่ม/จบและ transition ที่อนุญาต; อย่าสับสน action กับ state



Class (Samples: Final): attributes:type; links + multiplicity; inheritance → parent; composition ◆ ที่ whole; aggregation ◇; Samples ห้าม methods **Scenario**: basic = ปกติ; alternative = ทางเลือกที่ถูกต้อง; exception = ผิดพลาด; Activity ≠ Sequence; Use Case ≠ flowchart

Generic UI: List / Form / LoV

List: title/user/role, search/filter/sort, columns ตาม req, select/open/create, pagination, empty/loading/error; **Form**: header + line items + total/status + controls **สัญลักษณ์สอบ**: * นำหน้าช่องบังคับ; textbox แก้ได้; ขีดเฉียง = read-only/calculated; เลือก FK ด้วย LoV; รายการย่อยมี add/ remove และ validation **ตามสถานะ**: ใน KU Registration, Draft แก้/Save/Submit ได้; Done ล็อก; แสดงผลรอ/สำเร็จ/ผิดพลาดตาม contract; อย่านำชื่อ state นี้ไปใช้ทุกระบบ **Design system**: สี/ตัวอักษร/ระยะ/components/grid/breakpoints/ contrast/keyboard/focus; ตารางยาวอาจเลื่อนแนวนอน ไม่เติม split payment/cancel/refund หากโจทย์ไม่ให้; ทุก field/ button/transition ต้องโยง req และสิทธิ์ได้

Case patterns จาก Lecture 4 / review

Case	สิ่งที่ต้องรักษาเมื่อวาด UML/UI
POS	Cashier/Customer; รายการสินค้า/ยอด; cash/card; card ไม่ผ่าน → retry/cash/cancel; สำเร็จ → receipt/stock
Hotel	Guest ค้นวัน/เมือง เลือกหลายห้อง/ประเภท จ่าย KBank → ลดห้องว่าง/ยืนยัน/email ถึง Guest และ Hotel; Hotel Admin ดูแลโรงแรมตน
Ticket	Customer สร้าง Requested พร้อม action อัตโนมัติ; CCE/Programmer เพิ่ม action ตาม state; action แก้/ลบไม่ได้; latest status/due อัปเดต; ปิดแล้วห้ามเพิ่ม
Grades	Instructor เลือก semester/course/sections ดึงรายชื่อ/กรอกเกรด; ครบ → Ready → submit → Submitted; อัปเดต Transcript/email; ใช้ SSO/Registration/Student Info/Email ภายนอก

ตรวจรูป: ตรง req; guard/ผลข้างเคียงครบ; กรณีผิดกลับแก้ได้; ไม่แต่ง state/สิทธิ์

Lecture 5 · Security Engineering

Security landscape / CIA

Actor: ผู้กระทำ; **Asset**: สิ่งที่ต้องปกป้อง เช่น บัญชี/รายงาน; **Attack surface**: จุดรับการโจมตี เช่น API/upload; **Trust boundary**: ขอบเขตความเชื่อถือ เช่น browser → server C: Confidentiality ป้องกันการเห็นข้อมูลโดยไม่มีสิทธิ์; I: Integrity ป้องกันการแก้ไขโดยมิชอบ; A: Availability ผู้มีสิทธิ์ยังใช้ระบบได้ **Secure by design**: least privilege, defense in depth, fail safe/default deny, ออกแบบเรียบง่าย แบ่งส่วน ลดจุดโจมตี; ค่าเริ่มต้นปลอดภัยและยังใช้งานได้

Authentication ≠ Authorization

Authentication (AuthN): ยืนยันว่าเป็นใคร; **Authorization (AuthZ)**: ตรวจว่าทำได้ไหม ตามผู้ใช้ การกระทำ ทรัพยากรและ state; ตรวจที่ backend ทุก request **RBAC**: สิทธิ์ตาม role; **ABAC**: ตามคุณลักษณะผู้ใช้/ทรัพยากร/บริบท; **ACL**: รายการสิทธิ์ต่อทรัพยากร; ตรวจ ownership เทียบตัวตนที่ server ยืนยัน ไม่เชื่อคำอ้างว่าเป็นเจ้าของจาก client **Password**: slow hash + salt เฉพาะแต่ละรหัส; ไม่เก็บ/บันทึก plaintext. Base64 แค่แปลงรูปแบบ ไม่ใช่ encryption; encryption ถอดคืนด้วย key; hash ทางเดียว ใช้ตรวจเทียบ **Basic**: ส่ง credentials แบบ Base64 ต้องใช้ HTTPS; **Digest**: ตอบด้วย hash/nonce เป็นวิธีเก่า ไม่แทน HTTPS **Session**: server เก็บสถานะ ส่ง session ID ผ่าน cookie; หมดอายุ/logout แล้วใช้ต่อไม่ได้; current-user คืน profile ไม่คืน secret **JWT**: token ที่ลงลายเซ็น ไม่ได้เข้ารหัสเนื้อหาเสมอ; ตรวจลายเซ็น/วันหมดอายุ; ต้องออกแบบเพิกถอนเอง; access อายุสั้น ส่วน refresh ต้องปกป้องและเพิกถอนได้ **API key**: secret ของแอป/บริการ; **OAuth2**: มอบสิทธิ์ให้แอปอื่น; **OIDC**: เพิ่มการยืนยันตัวตนบน OAuth2; **SSO**: sign-in ครั้งเดียวใช้หลายระบบ

OWASP / Vulnerability / Control

ช่องโหว่	การป้องกัน / การทดสอบเชิงลบ
Broken access	ตรวจ role/owner/state ที่ backend; เรียก API ข้ามคน/role → ปฏิเสธ ไม่รั่วข้อมูล
Auth failure	error กลาง, rate limit, salted hash, expiry; ลอง wrong/inactive/logout แล้วเข้าตรง
SQLi	parameterized query/ORM; allowlist ชื่อ field; `' OR '1'='1` ต้องเป็นข้อมูล ไม่ใช่ SQL
XSS	encode output, sanitise rich text; ไม่ใช้ HTML ที่ไม่ปลอดภัย; payload ต้องไม่รัน
CSRF	browser ส่ง cookie ข้าม site; ใช้ token/SameSite/ Origin check; ปฏิเสธ forged write
SSRF	server ดึง URL จาก input; allowlist ปลายทาง/บล็อก internal/metadata/loopback
Upload	server ตรวจชนิด/ขนาด/เนื้อหา; เก็บแบบรันไม่ได้/ ดาวน์โหลดปลอดภัย; script ปลอม PDF ต้องไม่รัน
Config/ secrets	HTTPS, secure cookie, CORS เคร่งครัด; ไม่เผย debug/.env; เปลี่ยน secret ที่รั่ว
Deps	patch/audit packages; จำกัดสิทธิ์ DB/network; ปกป้อง backups

ซ่อนปุ่มไม่ใช่การควบคุมสิทธิ์; frontend validation ช่วย UX; backend validation คุม trust boundary **Status**: 400 ข้อมูลผิด; 401 ยังไม่ยืนยันตัวตน; 403 ไม่มีสิทธิ์; 404 ไม่พบ/ปิดบังตามกติกา; 409 ขัดแย้ง; 500 ข้อผิดพลาดกลาง; log correlation ID ไม่ใช่ password/token **คำอธิบายเสริม:** เมื่อปฏิเสธ ให้ตรวจ error/status และข้อมูลธุรกิจไม่เปลี่ยน; audit log ยังเกิดได้ตามกติกา contract

คำอธิบายเสริม: ตัวอย่างตรวจสิทธิ์ผ่าน API ตรง

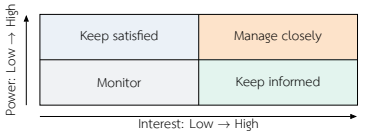
ทดสอบ	ผลที่ต้องตรวจตาม policy/contract
AuthN/gate	valid ผ่าน; wrong/inactive/ไม่ login ถูกปฏิเสธ; password gate ต้องคุม API ตรง
ผิด role	เรียก endpoint ตรงไม่ได้; ไม่มี business write
ข้าม owner	A เปลี่ยน ID ไปขอข้อมูล B → ไม่เผยข้อมูล/จำนวน/ไฟล์
logout/expiry	ใช้ session เดิมซ้ำไม่ได้ แม้เปิด URL/API ตรง
input/duplicate	error ปลอดภัย; DB ไม่ค้างครึ่งชุด; ข้อมูลเดิมไม่เสีย
log/config	ไม่พบ token/password/stack trace ใน response หรือ log ที่เปิดเผย

แนวตรวจ: AuthN → AuthZ → validation → ลงมือ/บันทึก; ไม่เชื่อ role จาก client; ลำดับย่อยขึ้นกับข้อมูลที่ต้องใช้ตรวจสิทธิ์ **Defense in depth**: ต่อไฟล์หลุด validation ต้องยังรั้นไม่ได้ และอ่านข้าม owner ไม่ได้

Lecture 6 · Project Initiation

Initiate → Plan → Execute → Close; เป็นวงจรโครงการ ส่วน SDLC อยู่ภายในโครงการ **Initiation 6**: goals, scope, deliverables, success criteria, stakeholders, resources → charter → อนุมัติ → วางแผน **Goal**: ผลที่ต้องการ;

Deliverable: สิ่งส่งมอบ; **Success criteria**: หลักฐานว่าสำเร็จ; launch = ส่งมอบ, landing = วัดผล **SMART**: Specific ชัด, Measurable วัดได้, Attainable ทำได้, Relevant สอดคล้อง, Time-bound มีเส้นตาย; ลดมัธยฐานเวลาตอบครั้งแรก 20% ภายในภาคนี้ด้วย routing อัตโนมัติ **OKR**: Objective = ทิศทาง; KRs = ผลที่วัดได้; ท้าทายและเชื่อมองค์กร → แผนก → โครงการ; KR ไม่ใช่รายการงาน ROI=(Gain-Cost)/Cost ×100%; Gain = ผลตอบแทน, Cost = ต้นทุนทั้งหมดในช่วงเดียวกัน; พิจารณาคุณค่าที่ดีราคาไม่ได้แยก G=10,000; C=6,000+25×12=6,300 → ROI=58.7% **Scope**: ตกลงสิ่งที่ทำ/ไม่ทำและข้อจำกัด; **Scope creep**: ขอบเขตเพิ่มโดยไม่ควบคุม ทั้งจากลูกค้าหรือทีม; ขอเปลี่ยน → วิเคราะห์ผล → เสนอทางเลือก → อนุมัติ → บันทึก **Triangle**: scope/time/cost; เมื่อตรึงด้านหนึ่งต้องปรับด้านอื่น ไม่รับงานเพิ่มโดยไม่ประเมิน **Stakeholders**: sponsor อนุมัติทุน, PM คุมแผน, ทีมทำ, ลูกค้าซื้อ, ผู้ใช้ใช้; เลือกวิธีดูแลตาม power/interest:

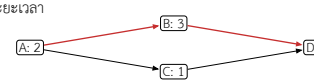


Charter: เหตุผล/เป้าหมาย/ขอบเขต/สิ่งส่งมอบ/เกณฑ์สำเร็จ/ทีม/ทรัพยากร/ทุน/กำหนดการ/สมมติฐาน/ความเสี่ยง/ผู้อนุมัติ **คำอธิบายเสริม:** เว็บที่ใช่ได้ = deliverable; ลดเวลารอ = goal; success criteria ระบุค่าตั้งต้น เป้าหมายและเส้นตายที่ตกลง

Lecture 6 · Project Planning

WBS / Schedule / Responsibility

WBS: project → milestone/deliverable → task → subtask; แยกแบบ top-down/bottom-up. **Milestone**: จุดสำเร็จ duration = 0; task มี owner และ duration **Gantt**: WBS no./ task/owner/start/due + แถบเวลา; ทำขนานได้เมื่อ dependency และทรัพยากรพร้อม; roadmap เน้น goal/ milestone ภาพรวม **Critical path (CP)**: เส้นทางงานที่พึ่งกันและใช้เวลารวมยาวสุด; **float**: เวลาที่เลื่อนได้โดยไม่เลื่อนวันจบ; CP มี float = 0 เมื่อกำหนดจบเท่ากับวันจบเร็วสุด เริ่มเวลา 0: EF=ES+d; ES=max(EF ของ pred); LS=LF-d; LF=min(LS ของ succ); Float=LS-ES **คำอธิบายเสริม:** สูตร/กราฟ: งานต่อกัน ไม่มี lag/ข้อจำกัดคน; ES/EF = เริ่ม/จบเร็วสุด, LS/LF = ช้าสุด, d = ระยะเวลา



A → B → D = 7 วัน (CP)
A → C → D = 5 วัน; C float = 2 วัน

Task	ES	EF	LS	LF	Float
A	0	2	0	2	0
B	2	5	2	5	0
C	2	3	4	5	2
D	5	7	5	7	0

วันที่แบบ inclusive: นับวันเริ่มเป็นวันที่ 1; Due = วันทำงานลำดับที่ d; งานถัดไปเริ่มวันทำงานถัดไป ไม่นับวันหยุด/ใช้คนข้อนเกิน กำลัง **Effort**: person-hours/days ≠ **duration**: เวลาที่งานใช้; Brooks: เพิ่มคนในงานที่ล่าช้าอาจยิ่งช้า PERT $E=(O+4M+P)/6$; $SD=(P-O)/6$; O ดีสุด, M น่าจะเป็น, P แย่สุด; E = ค่าคาดหมาย, SD = ส่วนเบี่ยงเบนมาตรฐาน; **คำอธิบายเสริม:** เมื่อประมาณการอิสระ $SD_{total}=\sqrt{\sum_i SD_i^2}$; ใช้หน่วยเดียวกัน **Buffer**: task buffer เผื่อ delay ที่คุมไม่ได้ เช่น รออนุมัติภายนอก; project buffer เผื่อรวมท้ายโครงการ; ไม่ใส่ทุก task ซ้ำซ้อน **RACI**: R ลงมือ, A รับผิดชอบผล, C ปรึกษาก่อน, I แจ้ง; ให้ A ชัดคนเดียว/งานและมี R; ไม่เกี่ยวใช้ “–”; ระวัง R กระจุกตัว **Risk**: ระบุ → วิเคราะห์โอกาส/ผลกระทบ → จัดลำดับ → ตั้งเจ้าของ → รับมือ → ติดตาม; **คำอธิบายเสริม:** risk อาจเกิดส่วน issue เกิดแล้ว **Kick-off**: เป้าหมาย/ขอบเขต/บทบาท/tools/ process/สิ่งต้องทำ; charter = ทำไม, plan = อย่างไร/เมื่อไร, risk register = อะไรอาจผิด

Lecture 7 · Execution / Agile / Scrum

MVP: ชิ้นเล็กสุดที่ทดสอบคุณค่าได้ → build-measure-learn → pivot/persevere; ไม่ใช่ phase 1 ที่ยังใช้ไม่ได้ **Pair**: driver เขียน/ navigator ตรวจ สลับกัน; **DevOps**: dev + ops ร่วมดูแล delivery ใช้ automation/feedback; Water-Scrum-Fall ยังติด batch/approval ที่ท้าย **PO**: คุณค่า/วิสัยทัศน์/เรียง backlog/ ตัดสิน req/ยอมรับ/ส่งมอบ; **SM**: อำนวยความสะดวก คุมเวลา แก้อุปสรรค ไม่สั่งแข่งงาน/ตัดสิน product แทน PO; **Developers**: จัดการตนเอง ข้ามทักษะ เลือกวิธีทำ/คน/capacity **Artifacts 3**: product backlog, sprint backlog, Done increment; **Events 5**: Sprint, Planning, Daily, Review, Retrospective; p. 47 นับ refinement แทน Sprint ในหนวด meetings

Story / Backlog refinement

As a <role>, I need <function>, so that <value>; role ไม่ใช่ชื่อคน; มี description/details/assumptions/AC Given <precondition> When <event> Then <observable outcome>; And/But เสริม; ทดสอบบวก/ลบ ไม่กำหนด implementation เกินความต้องการ **INVEST**: Independent, Negotiable, Valuable, Estimable, Small, Testable; **Epic**: ใหญ่เกิน sprint ต้องแยกก่อน **Groom/refine**: ปฏิเสธ/พักใน icebox/จัด backlog; PO เรียงคุณค่า ทีมแยกและชี้แจงงานล่วงหน้าราว 2 sprints; DoR = พร้อมเริ่ม **Metadata**: Assignees, Labels (ชนิด), Milestone (sprint), Estimate (points); enhancement เพิ่มพฤติกรรม, bug แก้ผิด, technical debt ปรับโครง/refactor/env/dependencies

Sprint planning / execution

Points: effort/complexity/uncertainty แบบสัมพัทธ์; Fibonacci 1,2,3,5,8,13,21; anchor 3: +1 ขึ้น = 5, +2 = 8, −1 = 2; ไม่ใช่ชั่วโมง/ไม่เทียบข้ามทีม **Velocity**: points ที่ทีมทำเสร็จต่อ sprint; ใช้ประวัติร่วมกับกำลังทีม (capacity); เลือกตาม goal, priority, dependencies และ ∑points ≤ capacity; ไม่เติมงานเพียงเพราะยังมีที่ **Planning**: PO ให้ goal/ลำดับ; ทีมประเมินและเลือกงาน/วิธีทำ; SM คุมเวลา; sprint backlog = forecast + plan; มักยาว 2 สัปดาห์ **Board**: New → Icebox/Backlog → Sprint Backlog → In Progress → Review/QA → Done; WIP 1/คน เว้นติดขัด; เลือกงานสำคัญสุด; เสร็จ → PR → merge/ Done ตามกติกา **Daily**: ตามสไลด์ 15 นาที คุยงานเมื่อวาน วันนี้ และอุปสรรค; แก้รายละเอียดหลังประชุม ไม่ใช่รายงานหัวหน้า **Review**: demo/AC กับ PO/ผู้เกี่ยวข้อง; feedback → Issue; **Retro**: ทีม/SM ทบทวนดี/ไม่ดี/สิ่งที่จะเปลี่ยน แล้วนำไปปรับ sprint หน้า **Burndown**: x = เวลา, y = points คงเหลือ; ลดเมื่อ Done ไม่ใช่เริ่มทำ; metrics ต้องมี baseline และนำไปปรับงานได้ **จบ sprint (pp. 161-165)**: งานไม่เริ่มคืน backlog/ถอน milestone; งานค้างปรับ story/points ส่วนที่ทำ ติด unfinished และแยกงานที่เหลือ; เครดิตตามสไลด์ ≠ ผ่าน DoD **Docs**: req/ design/technical/QA/user/SOP; **Org**: traditional หัวหน้าเดียว, matrix หลายสาย; Conway: โครงที่มสะท้อนระบบ

Metrics: ความหมาย ไม่ใช่แค่ชื่อ

Metric	ใช้ตัดสินใจอะไร
Mean Lead Time	เวลา idea → production; ยาว → หาเวลารอ/คอขวด
Release frequency	ส่งขึ้นใช้งานบ่อยแค่ไหน; batch ใหญ่ทำให้ feedback ช้า
Change failure rate	สัดส่วน release/change ที่ก่อปัญหา; ไม่ใช่จำนวน tests ที่ผ่าน
MTTR	เวลาเฉลี่ยฟื้นระบบหลัง failure; ดูทั้งตรวจพบและกู้คืน

คำอธิบายเสริม: ระบุจุดวัด หน่วย ช่วงเวลาและ baseline; จำนวน commits/บรรทัดโค้ด ≠ คุณค่า

วิศิษฐ์ สุวรรณเนาว์ 67070501042

Practice Midterm · Part 1

Part 1 รวม 25 คะแนน: ข้อกำหนด ข้อเสนอโครงการ RBAC และความปลอดภัย; ไม่ใช่ข้อวิเคราะห์ข้อกำหนดข้อเดียว

1) วิเคราะห์ข้อกำหนดให้ครบ 5 ขั้น

จำแนกชนิด → อธิบายจุดประสงค์ → ประเมินคุณภาพพร้อมเหตุผล → วิธีตรวจ/ผลที่คาด → เสนอแก้; แยกข้อเท็จจริงจากข้อเสนอ

Workshop req	แนววิเคราะห์
Browse open	FR; นักศึกษาที่ login แล้วดู workshop ที่เปิดรับ; เครียม open/closed → เห็นเฉพาะ open
“Quick”	NFR; กำกวมเพราะไม่มีตัวชี้วัด/ภาระงาน ต้องตกลงก่อนทดสอบ
≤3/semester	BR/domain; ทดสอบมี 2 เพิ่มเป็น 3 ได้; มี 3 แล้วเพิ่มครั้งที่ 4 ต้องปฏิเสธ
Cancel + list	รวมสอง FR; แยกนักศึกษายกเลิกกับผู้ประสานงานดูรายชื่อ; ถามขอบเขตสิทธิ์ให้ชัด

คำอธิบายเสริม: เสนอ NFR: เมื่อมีผู้ใช้พร้อมกัน 100 คน คำขอสมัครที่ถูกต้องอย่างน้อย 95% ได้รับการยืนยันภายใน 2 s; ไม่ใช่ค่าที่โจทย์ให้ ต้องตกลงจุดวัด เครื่องข่าย ชุดข้อมูลและกรณีผิดพลาด **FR template**: The system shall allow <actor> to <action/object> when <condition>. ระบุผลและข้อยกเว้นที่ตรวจได้

quick/easy/reliable ยังไม่พอ; อย่าตั้งตัวเลขแล้วอ้างว่าลูกค้าให้มา; test ต้องมี input/state, action, expected result ไม่ใช่แค่ “ทำ unit test”

2) WorkshopHub proposal: 8 ช่อง

Element	หลักฐาน / สิ่งที่ขาด
Problem	หลายฟอร์ม รวบรวมด้วยมือ → ข้อมูลซ้ำ/ล่าช้า
Users	ระบุ Student/Coordinator; ต้องถามผู้อนุมัติ
Solution	เว็บดู workshop/สมัคร/จัดการผู้เข้าร่วม
Value	ลดข้อผิดพลาด/งานธุรการ; ขาด metric/baseline
Scope	รุ่นแรกสมัครฟรี; payment/certificates ไว้ภายหลัง
Risk	ไม่ระบุ; เสนอเรื่องซ้ำซ้อน/สมัครพร้อมกัน/privacy
Effort	“ก่อนภาคหน้า” เป็น deadline ไม่ใช่ effort estimate
Evidence	ยังไม่มี AC/test/demo/sign-off

ย่อหน้าแก้ไข: ปัญหา/ผู้ใช้ → solution/ขอบเขตรุ่นแรก → คุณค่า/metric ที่เสนอ → risk/วิธีรับมือ → effort/ทรัพยากร/deadline → หลักฐาน/ผู้ยอมรับ เสนอให้ Coordinator ลอง สมัคร/ยกเลิก ตรวจข้อมูลซ้ำและสิทธิ์; demo + tests ผ่าน + sign-off; ประเมิน person-days กับทีมก่อนรับปาก ไม่แต่งตัวเลขเป็นข้อเท็จจริง

คำอธิบายเสริม: ตัวอย่างย่อหน้าเสนอปรับปรุง

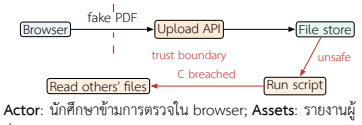
WorkshopHub จะแทนฟอร์มแยกที่ผู้ประสานงานต้องรวบรวมเอง ซึ่งทำให้ข้อมูลซ้ำและยืนยันล่าช้า นักศึกษาดูและสมัคร workshop ฟรีได้ ผู้ประสานงานจัดการรายชื่อ รุ่นแรกไม่รวมการชำระเงินและไประกาศ เสนอให้ตกลงเป้าลดข้อผิดพลาด/เวลาธุรการเทียบค่าตั้งต้น ประเมินความเสี่ยงสมัครซ้ำพร้อมกันและข้อมูลส่วนบุคคล แล้วตรวจด้วยกฎบน server/สิทธิ์ แยกงานเพื่อประเมินแรงงานและยืนยันว่าจะทันก่อนภาคหน้า การรับงานควรใช้ tests ที่โยง AC การสาธิต และผู้ประสานงานยอมรับ **แยกที่มา**: ปัญหา ผู้ใช้และขอบเขตมาจากโจทย์; ตัวชี้วัด ความเสี่ยงและวิธีรับงานเป็นข้อเสนอ ยังไม่ถือว่าลูกค้าตกลง

3) RBAC: default deny / ไม่สืบทอดสิทธิ์

Operation	Student	Coordinator
View published workshops	Allow	Allow
Submit registration	Allow	Deny
Create workshop	Deny	Allow
Edit workshop	Deny	Allow
Export participant list	Deny	Allow

Coordinator ไม่ได้สิทธิ์สมัครเองโดยอัตโนมัติ; คนละ 1 role; ไม่ grant = deny; อย่าเพิ่มเงื่อนไข owner ที่โจทย์ไม่ให้ **Backend**: ยืนยันตัวตน → ตรวจ role/operation → ตรวจข้อมูล → ทำงาน; เพิ่ม owner/state เฉพาะที่ policy กำหนด **คำอธิบายเสริม:** Negative test: Student เรียก `POST /workshops` ตรง → 403 และไม่เพิ่มข้อมูล; Coordinator ขอสมัครต้องถูกปฏิเสธ; endpoint/status นี้เป็นแบบออกแบบเพิ่ม

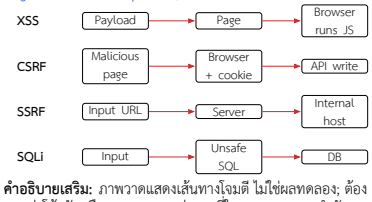
4) Upload attack: ข้าม trust boundary



Actor: นักศึกษาข้ามการตรวจใน browser; **Assets**: รายงานผู้อื่น/ระบบ; **Surface**: upload API/file URL; **Boundary**: client → server และไฟล์ข้อมูล → โค้ดที่รัน **CIA**: อ่านรายงานคนอื่นละเมิด C; code execution เสีย I/A ด้วย แยกเหตุที่เกิดแล้วจากผลที่อาจเกิด **Control/test 1**: server ตรวจ PDF type/content/size; ส่ง script, MIME ปลอมหรือไฟล์ใหญ่ → 4xx ไม่เก็บเป็นรายงาน **Control/test 2**: เก็บนอก webroot/รันไม่ได้; download handler ตรวจ owner; เปิด script → ไม่รัน; ขอรายงานคนอื่น → ปฏิเสธ **คำอธิบายเสริม:** สุ่มชื่อไฟล์ ตรวจ path จำกัดสิทธิ์ สแกนมัลแวร์ตามความเสี่ยงและ log อย่างปลอดภัย; ตรวจใน browser ช่วย UX ไม่ได้แทน server

ตอบ security พร้อมเหตุผล

ช่องโหว่ → จุดเข้า/asset → boundary → CIA → defense → negative check + expected; ไม่ใช่แค่รายชื่อ attack



คำอธิบายเสริม: ภาพวาดแสดงเส้นทางโจมตี ไม่ใช่ผลทดลอง; ต้องบอกว่าโค้ดรันหรือ request ถูกส่งจากที่ใด **Boundary**: จำกัด 3 → ทดสอบ 2/3/4, ข้ามภาค, ยกเลิกแล้วมีไหม? เงื่อนไขที่โจทย์ไม่บอกต้องถาม ไม่เพิ่มกฎเอง **แยกศัพท์**: threat = เหตุร้ายที่อาจเกิด; vulnerability = จุดอ่อน; attack = การใช้จุดอ่อน; risk ประเมินโอกาส/ผลกระทบ; control = มาตรการลดความเสี่ยง

Practice Midterm · Part 2

Part 2 รวม 25 คะแนน: WBS/RACI/Gantt ใช้บทบาท PM; grooming ใช้ PO; planning ใช้ SM; อธิบายตามบทบาท ไม่ตัดสินแทนทีม

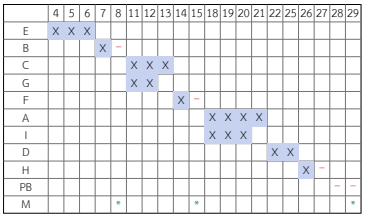
1) DormFix: WBS 3 + RACI 2 + Gantt 5

WBS: 1.1 Requirements approved: E → B; 1.2 Designs approved: (C||G) → F; 1.3 System accepted: (A||I) → D → H; เว้น 1.4 ไม่เติม milestone เอง **Dependency**: E เขียน req → B อนุมัติ; C ออกแบบจอ/G ออกแบบ DB → F อนุมัติ; A ฟอร์ม/I dashboard → D ทดสอบทั้งคู่ → H ยอมรับ

Task	PM	Aom	Bank	Chai	Dao
E	A	R	C	I	–
C	A	C	R	I	–
G	A	–	C	R	I
D	A	I	–	C	R

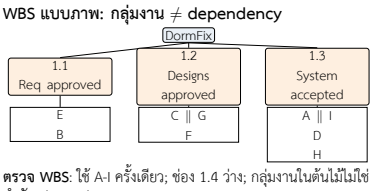
คำอธิบายเสริม: ตาราง RACI นี้มาจาก meeting notes ที่เราแต่งใน mock exam ไม่ใช่เฉลยตายตัวของอาจารย์; ข้อสอบจริงให้อ่าน notes ก่อนลงตัวอักษร **Gantt (Jan 2027)**: เริ่ม จ. 4; ทำเต็มเวลา จ.-ศ.; B/F/H รออนุมัติภายนอก เผื่อ task buffer 1 วัน; project buffer 2 วันท้าย

ID	Owner	d	Work dates (+buffer)
E	Aom	3	4-6 Jan
B	PM	1	7 Jan (+8 Jan)
C	Bank	3	11-13 Jan
G	Chai	2	11-12 Jan
F	PM	1	14 Jan (+15 Jan)
A	Chai	4	18-21 Jan
I	Dao	3	18-20 Jan
D	Dao	2	22,25 Jan
H	PM	1	26 Jan (+27 Jan)
PB	–	2	28-29 Jan



M: milestone วันที่ 8/15/29 Jan; X = งาน, – = buffer, * = milestone ไม่เพิ่มวันทำงาน; Start/Due นับรวม buffer; ลูกศร dependency ชี้งานที่รอ เหตุผล: B/F/H รอคนนอกที่มีงานอื่น; ทีมภายในใช้ buffer รวม 2 วันท้าย; C||G และ A|| ขนานได้; milestone ใช้เวลา 0 วัน

WBS แบบภาพ: กลุ่มงาน ≠ dependency



ตรวจ WBS: ใช้ A-I ครั้งเดียว; ช่อง 1.4 ว่าง; กลุ่มงานในต้นไม้ไม่ใช่ลำดับ dependency

2) Backlog grooming (10): Issue #12

ในฐานะนักศึกษา ฉันต้องการจองสนามแบดหนึ่งชั่วโมง เพื่อไม่ต้องต่อคิวที่เคาน์เตอร์ (ใช้โครง As a / I need / So that) **รายละเอียด**: มีสนาม/เวลาเปิดแล้ว; นักศึกษาถือการจองได้คนละหนึ่งรายการต่อวัน; Court 2 วันนี้ 17:00 ว่าง

Given Court 2 วันนี้ 17:00 ว่าง

When นักศึกษาจอง Court 2 เวลา 17:00

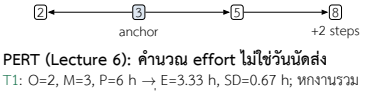
Then ช่วงนั้นแสดง “Booked” **คำอธิบายเสริม:** AC ลบ: มี booking วันนี้อยู่แล้ว → ขอเพิ่ม → ปฏิเสธและไม่สร้างรายการ; ถามเงื่อนไขยกเลิกก่อนขยายภฏ **Metadata หลังกำหนดรายละเอียด**: Assignees –; Labels enhancement; Milestone –; Estimate –. ยังไม่ใช่ Sprint planning ไม่กำหนดคน/points/sprint เอง **Bug variant (Issue #15)**: ยกเลิกแล้วยัง Booked → label bug; Given Court 1 เวลา 18:00 ถูกจอง; When ยกเลิก; Then แสดง “Free” ตามโจทย์ **คำถามที่ควรถาม**: วัน/เวลาของการจองใช้เขตเวลาใด? ยกเลิกได้ไหมและคืนสิทธิ์วันนั้นหรือไม่? รายละเอียดที่ยังไม่ตกลงไม่เดิมเป็น AC บังคับ

3) Sprint planning (5): Goal + capacity

ShuttleNow: goal = ติดตามรถ/เวลาถึง; anchor 3, velocity 16

#	Story / เทียบ anchor	Pt	Pick
1	Live position (+2 ชั้น)	8	Yes
2	QR fare (เท่ากัน)	3	No
3	Arrival at my stop (+1)	5	Yes
4	Arrival alert (เท่ากัน)	3	Yes
5	Arrival at all stops (+1)	5	No
6	Fix mobile truncation (−1)	2	No

เลือก 1/3/4: 8+5+3=16; 2 ไม่ตรง goal; เพิ่ม 5 รวม 21 เกิน capacity; 6 ไม่มีที่เหลือ. Fibonacci นับขั้น ไม่บวกเลขตรง



PERT (Lecture 6): คำนวณ effort ไม่ใช่วันนัดส่ง

T1: O=2, M=3, P=6 h → E=3.33 h, SD=0.67 h; ทางรวม E=35.67 h, SD=3.23 h เมื่อประมาณการอิสระ → 35.7 ± 3.2 h (หนึ่ง SD) ไม่ใช่ช่วงที่รับประกัน; วันจบต้องดูงานขนาน/คนด้วย

Git: คำสั่ง remote ที่ควรรู้

```
git fetch origin
```

รับ commits/อัปเดต origin/*; ไม่เลื่อน local branch, HEAD หรือ working tree ในการ fetch ปกติ

```
git switch main
git merge --ff-only origin/main
```

อัปเดต main แบบ FF; ถ้าแยกสายจะปฏิเสธ ไม่สร้าง merge commit

```
git push origin sprint_1_staging
```

เผยแพร่ staging/อัปเดต tracking ref เมื่อสำเร็จ; push feature ไม่เผยแพร่ merge ที่อยู่เฉพาะ staging

Practice Midterm · Part 3

UML/UI รวม 30 คะแนน: เลือก 3 แบบจาก Sequence, Activity, UI, Use Case, State แบบละ 10; mock ไม่ได้ยืนยันว่าจะออก 3 แบบใด

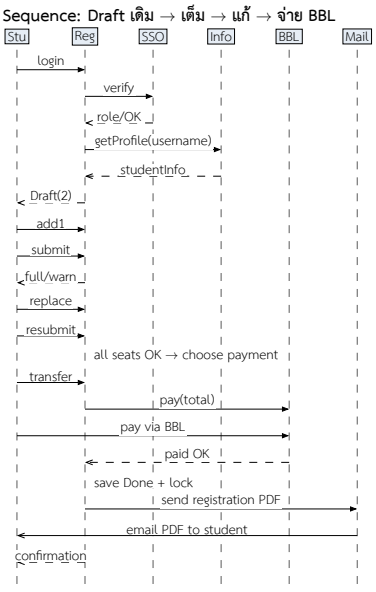
KU Registration: อ่าน R1-R11 ก่อนวาด

SSO คืน role; ใช้ภาคปัจจุบัน/Student Info; เพิ่มหลาย offering แล้วยอดเปลี่ยน; Draft บันทึก/แก้ได้; ตรวจทุก section; เต็ม → แก้/ส่งใหม่; card → KBank, transfer → BBL; สำเร็จ → Done/ล็อก/email PDF

Actors (R1-R4)	ขอบเขตที่ระบุในโจทย์
Department Admin	สร้าง course; ตั้ง offerings: semester/code, section/room/time/notes/max students
University Admin	สร้าง semester และกำหนด active semester

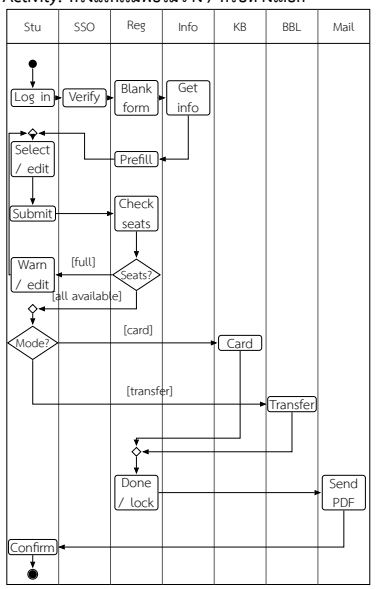
SSO คืน Instructor ได้ แต่ R1-R11 ไม่กำหนดงานให้; ไม่แต่งสิทธิ์ **คำอธิบายเสริม:** ภาพวาดย่อจากข้อกำหนด ข้อความไม่ใช่ข้อ API จริง

Sequence: Draft เดิม → เต็ม → แก้ → จ่าย BBL



เริ่มจาก Draft เดิม 2 offerings แล้วเพิ่ม 1; ส่งแล้วเต็มก่อนแก้/ส่งซ้ำสำเร็จ; Reg จัดการ DB ภายใน; หากโจทย์ให้ DB lifeline ต้องแสดง call

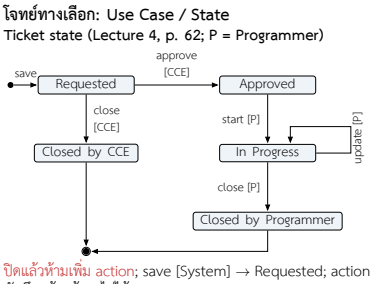
Activity: ครั้งแรกเริ่มฟอร์มว่าง / ครบทางเลือก



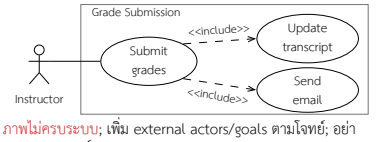
KB = KBank; BBL = Bangkok Bank; Info = Student Info; จ่ายสำเร็จตามโจทย์ วนเฉพาะแก้ section เต็ม

โจทย์ทางเลือก: Use Case / State

Ticket state (Lecture 4, p. 62; P = Programmer)



ปิดแล้วห้ามเพิ่ม action; save [System] → Requested; action บันทึกแล้วแก้/ลบไม่ได้ **Grades**: submit เฉพาะ Ready → Submitted; ปรับ transcript/ส่ง email (Use Case ย่อด้านล่าง)



ภาพไม่ครบระบบ; เพิ่ม external actors/goals ตามโจทย์; อย่าย้าย state/สิทธิ์ระหว่าง Registration กับ Grades **เช็ก UML**: decision มี guard; merge รวมทางเลือก; fork/join แยก/รอทางขนาน; Sequence แสดงผู้ส่ง/ผู้รับ; State ใช้ event [guard] / effect

Practice Midterm · Part 3 (ต่อ)

UI ต่อ / Git รวม 20 คะแนน

คำอธิบายเสริม: แบบหน้าจอย่อจาก R5-R11; ค่าตัวอย่างไม่ใช่ข้อกำหนด และปุ่มที่เพิ่มต้องสอดคล้องกับสิทธิ์

Course Offering List — Student

Semester: active ///// Course: [search]

Code	Sec	Cr	Fee	Seats
CPE334	1	3	8,000	3

Detail: ID, name, room, time, notes, semester, maximum, number registered

[Select / Add to Form] [Prev] 1 [Next]

Student Registration Form

[Close] [Save Draft] [Submit]

Semester: active /////

Student ID / name / username /////

Degree programme / email /////

Total fee ///// Status: Draft /////

*Offering [LoV] | Code / Sec / Cr / Fee /////

113 [select] | CPE334 / 1 / 3 / 8,000

[+ Add line] [Remove line]

Full section → warning → edit / resubmit

Done → locked; confirmation + emailed PDF

/////: ขีดเฉียงแทน read-only; *Offering เป็น FK ที่แก้ได้ เลือกด้วย LoV; ข้อมูลที่ดึงมา/total/semester/student อ่านอย่างเดียว; แสดง R7 ครบผ่าน columns/detail ไม่ต้องยัดทุก field ลงตารางเดียว **ตรวจ UI**: filters/selection, header/line items, add/remove, total, state, permissions, validation; เต็ม → แก้/ลองใหม่; Done → แก้ไม่ได้

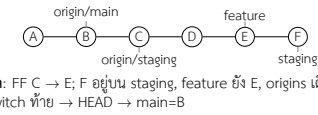
Git: graph = ancestry + pointers

HEAD → current branch → commit; เมื่อ clean จะได้ working tree ตาม HEAD; origin/* คือ remote-tracking ref สถานะ remote ที่ local รู้ล่าสุด ไม่ใช่ GitHub แบบสด; tag ชี้ commit คงที่ **switch**: เปลี่ยน HEAD/working tree ไม่เลื่อน branch; **commit**: สร้างลูกของ HEAD แล้วเลื่อน branch ปัจจุบัน; **merge**: รวมเข้า branch ปัจจุบัน ไม่เลื่อน source **FF**: current เป็น ancestor ของ target → เลื่อน current ไม่สร้าง commit; **3-way**: แยกสาย → commit ใหม่ 2 parents; graph ไม่บอกว่าจะมี file conflict

คำอธิบายเสริม A: FF → commit → switch

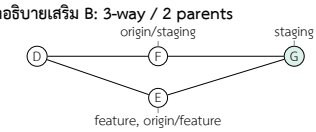
เริ่ม main=B, staging=C, feature=E บน C → D → E; origin/main=B, origin/staging=C

```
git switch sprint_1_staging
git merge feature/load_scores
git add score_data.py scores.csv
git add validation_notes.txt
git commit -m \
"Document score validation cases"
git switch main
```



ผล: FF C → E; F อยู่บน staging, feature ยัง E, origins เดิม; switch ท้าย → HEAD → main=B

คำอธิบายเสริม B: 3-way / 2 parents



`git merge feature/basic_statistics` บน F → G มี parents F/E; switch main ท้าย → HEAD=B; ภาพขยาย fork ที่ D; main/origin/main=B เป็น ancestor ของ D; source/origins ไม่เลื่อน

Sample 2.8: สร้าง feature จากฐานหลัง FF

หลัง FF จาก C → E บน staging จึงสร้าง branch ใหม่ที่ E; `branch` อย่างเดียวยังไม่เปลี่ยน HEAD

```
git branch feature/grade_distribution
git switch feature/grade_distribution
```

แก้ `report.py`, `scores.csv`; สร้าง `grade_ranges.csv` แล้ว stage เฉพาะสามไฟล์/commit F → HEAD อยู่ feature ใหม่=F; staging และ feature/pass_rate ยัง E, origin/staging ยัง C **Inspect**: `status` ดู untracked/index; `diff` ดู unstaged; `diff --cached` ดู staged; sample ห้ามใช้ `git add .` **วาดให้ครบ**: commits/parents, branch/origin ทุกตัว, tag, HEAD; merge มี 2 parents; commit ที่มีเฉพาะ local คือส่วนที่ ahead จาก tracking ref

Explain state: fetch / tag / unpublished merge

Sample 2.6: A → B → H; main=B, origin/main=v1.0=H; switch main ได้ B; FF origin/main ได้ H; staging/feature/tag ไม่เลื่อน **Sample 2.7**: staging=H (parents E/G), origin/staging=E, feature=G; push feature ไม่เผยแพร่ H; ต้อง `git push origin sprint_1_staging` → origin/staging=H เมื่อสำเร็จ

Sample 2.9: อัปเดต staging ก่อนรวม feature

เริ่ม staging=D, origin/staging=G; D → E → F อยู่บน feature และ D → G แยกอีกสาย

```
git switch sprint_2_staging
git merge --ff-only \
origin/sprint_2_staging
git merge feature/export_results
git push origin sprint_2_staging
```

วาดที่ละขั้น: switch → HEAD ที่ D; FF → staging=G; merge → H มี parents G/F; push → origin/staging=H; feature ยัง F และ main ไม่เลื่อน ห้าม FF feature ที่แยกสาย; FF ใช้เฉพาะ D → G; ไม่รวม remote ที่ยังไม่ได้ fetch โดยเดาจาก GitHub

วิศิษฐ์ สุวรรณเนาว์ 67070501042